

Competitive Programming

Section 38: Static Queries and Binary Lifting

Competitive Programming Course

2026-07-06

Table of contents

1	Static query preprocessing	3
2	Sparse table worked query	4
3	Binary lifting	5
4	Recognition	6
5	Local exercises	7
6	How to use the checker	8

1 Static query preprocessing

When the data never changes, preprocessing can replace a dynamic data structure. Sparse tables answer idempotent range queries in $O(1)$ after $O(n \log n)$ preprocessing.

An operation is idempotent when:

```
op(x, x) = x
```

Minimum and gcd are idempotent. Sum is not.

Store:

```
st[k][i] = answer for range [i, i + 2k - 1]
```

Build by joining two half-length blocks:

```
st[k][i] = op(st[k-1][i], st[k-1][i + 2(k-1)])
```

A query $[l, r]$ has length len . Let $k = \text{floor}(\log_2(len))$. Two overlapping blocks of length 2^k cover the range:

```
[l, l+2k-1] and [r-2k+1, r]
```

Overlap is fine because the operation is idempotent.

2 Sparse table worked query

Array:

index:	1	2	3	4	5	6
value:	5	1	4	7	2	9

Query minimum on $[2,5]$. The length is 4, so $k = 2$ and one block of length 4 exactly covers the range:

```
st[2][2] = min([2,5])
```

Query minimum on $[2,6]$. The length is 5, so $k = 2$, block length 4. Use:

```
[2,5] and [3,6]
```

They overlap, but $\min(x,x)=x$, so overlap does not change the answer. For sums this would double-count the overlap, which is why sparse tables do not answer ordinary range sums this way.

3 Binary lifting

Binary lifting stores jumps of length powers of two:

```
up[k][v] = 2k-th ancestor of v
```

Recurrence:

```
up[k][v] = up[k-1][ up[k-1][v] ]
```

To move up d steps, decompose d in binary and take the corresponding jumps. This is the same power-of-two idea as sparse tables, applied to paths instead of intervals.

Example:

```
d = 13 = 8 + 4 + 1
```

To move a node up by 13, take jumps:

```
up[3], then up[2], then up[0]
```

The order does not matter for pure ancestors as long as every selected bit is used exactly once.

Minimal C++ jump:

```
int jump(int v, int d) {
    for (int k = 0; d; k++, d >>= 1) {
        if (d & 1) v = up[k][v];
        if (v == -1) break;
    }
    return v;
}
```

Minimal Python jump:

```
def jump(v, d):
    k = 0
    while d and v != -1:
        if d & 1:
            v = up[k][v]
        d >>= 1
        k += 1
    return v
```

4 Recognition

Static range min/gcd/max? Sparse table.
Static range sum? Prefix sums instead.
Repeated ancestor jumps? Binary lifting.
Need updates? Segment tree or Fenwick, not sparse table.

5 Local exercises

- A. Static Range Minimum: Given an immutable array, answer minimum queries on subarrays.
- B. Static Range GCD: Given an immutable positive array, answer gcd queries on subarrays.
- C. Kth Ancestor: In a rooted tree with root 1, answer queries asking for the k -th ancestor of a node, or -1 if it does not exist.

6 How to use the checker

The local checker compiles/runs both languages and compares output tokens exactly:

```
CP_TARGET=solution python3 sections/38_static_queries_binary_lifting/check.py
```

Replace `CP_TARGET=solution` by your own `solve.cpp` or `solve.py` attempts when practicing.